

数据架构演进

从小蚕 StarRocks 存算一体到 Flink 双出 + MySQL 服务层

现状：StarRocks 被当成全能层

StarRocks 同时承接 OLAP (分析) + OLTP (点查) + 存储，后端服务直接查 StarRocks 时延迟不可控。

当前问题架构

业务数据源 → StarRocks 存算一体 (单集群) → 看板 / 报表 / 后端服务
后端 OLTP 点查也走 StarRocks → 高峰期 3 秒超时，I/O 不可隔离

存算一体在腾讯云上的隔离困境

Workload Group 只能做软隔离

CPU 和内存可以按 query 分组划分，但磁盘 I/O 和网络带宽是所有 Workload Group 共享的。后端点查扫热数据时吃 I/O，分析侧大查询 GROUP BY 全表扫描也吃 I/O —— 两股流量在同一个 BE 进程里抢同一块盘，谁也躲不开。

扩容绑死

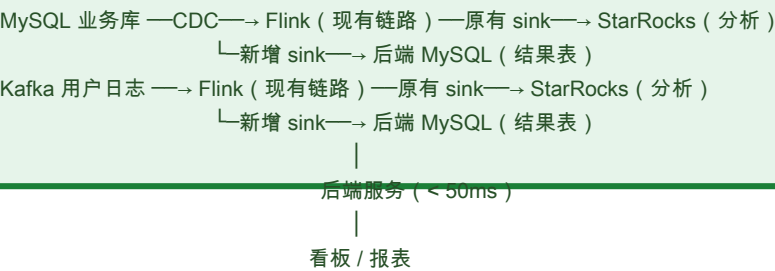
想给分析侧加算力就必须同时加存储 (每扩一个 BE 都带一块盘)，想扩存储容量也得捎上计算。成本不是线性增长的，是按节点整体翻倍的。高峰期分析查询和点查并发上去，只能整体升配，没法单独给某一层加弹性。

实践验证

此前试图用「再搭一套 StarRocks 集群给后端专用」来缓解，结果是：存储数据要写两份，两套集群各自维护，单集群内部的 I/O 竞争依然存在，本质上是用冗余成本换一个不彻底的解法。

改造方案：Flink 双出 + MySQL 服务层

改造方案 — 不动链路，只加 sink



Flink 已经在跑，后端 MySQL 本来就有，改造只做两件事：1) Flink 作业加一个 MySQL JDBC sink，把后端需要的结果表写到后端 MySQL；2) 后端服务把查询切到 MySQL。不新增组件，不改业务代码。

各层职责

层	角色
Flink	数据中转与计算。消费 binlog 和 Kafka，清洗聚合后双出到 StarRocks 和 MySQL
StarRocks	分析查询。只接 Flink 写入，不再直接扛后端点查
后端 MySQL	扛住后端 OLTP 点查。Flink 把预计算好的结果表写到这里
后端服务	只访问 MySQL，延迟 < 50ms

跟现状的核心区别

现状	改造后
Flink 只出 StarRocks	Flink 双出：StarRocks + MySQL
StarRocks 扛点查	MySQL 扛点查
后端服务查 StarRocks	后端服务查 MySQL
一套集群扛所有负载	分析（StarRocks）和服务（MySQL）分层
后端 3 秒超时	后端 50ms 以内
再搭一套 SR 做隔离，冗余成本翻倍	不新增组件，Flink 多一个 sink

成本分析

- 改造几乎没有增量成本——Flink 和 MySQL 都是现有组件：
- Flink 加一个 sink 不增加 TaskManager，写入量不变
 - 后端 MySQL 加几张结果表，存储增量很小（只存后端要查的列，不存全量明细）
 - 不引入 OSS / Paimon / Redis / StarRocks CN 等新组件

跟此前两套 StarRocks 集群方案或 Paimon 路线比，这条路没有新组件开销，只有一次性的 Flink 作业改造和结果表建表工作。

边际成本对比

场景	当前	Flink 双出后
分析查询增加	扩 BE 节点，磁盘一起买	StarRocks 只跑分析，扩容不变但节奏可控
后端点查压力增大	扩 BE 或加集群，I/O 竞争仍在	扩 MySQL 读副本，成本远低于扩 BE
闲时降本	缩不下来（BE 带着数据）	StarRocks 分析侧可考虑缩容
新增数据链路	新链路从头建	往 Flink 上加 sink 即可

后续演进：什么时候加 Paimon

Flink 双出解决的是当前最紧迫的问题——后端点查超时。Paimon + StarRocks 存算分离解决的是另一个层次的问题：数据量到几十 TB 后存储成本压不住、需要多引擎共享同一份数据、需要 Time

Travel 做数据回溯。

当前阶段不必上 Paimon，但 Flink 双出的架构天然兼容后续演进——StarRocks 那路的数据已经管好了，以后如果要切到 Paimon 做湖格式，只需要把 StarRocks 的写入目标从内部表改成 Paimon 外表，Flink → MySQL 这条服务链路完全不受影响。

加 Paimon 的触发条件：

- 数据量到几十 TB，StarRocks 本地存储成本成为主要矛盾
- 有多引擎共享同一份数据的需求（StarRocks + Flink + Spark 都要读同一份）
- 需要 Time Travel 做数据回溯（财务对账、历史快照）
- 有专门的数据平台团队能维护 Paimon 集群

结论：Flink 加一个 MySQL sink，后端服务切到 MySQL，3 秒超时直接解决。不动链路、不建新集群、不加组件，现有条件